Three Number Sort

AlgoExpert · Medium · Sorting

The problem

- Sort an array in place according to a 3-value `order` (Dutch national flag).

Intuition

- Dutch national flag: three regions grown by three pointers in one pass.

Brute force vs optimal

- Sort → O(n log n)
- Counting pass + rewrite → O(n), two passes
- Dutch national flag → O(n), one pass, O(1) space

Approach

- 1. First, second and third pointers
- 2. Swap firsts to the front
- 3. Leave seconds
- 4. Swap thirds to the back

Worked example




```
saw 1: second → leave it
```

1: second → leave it

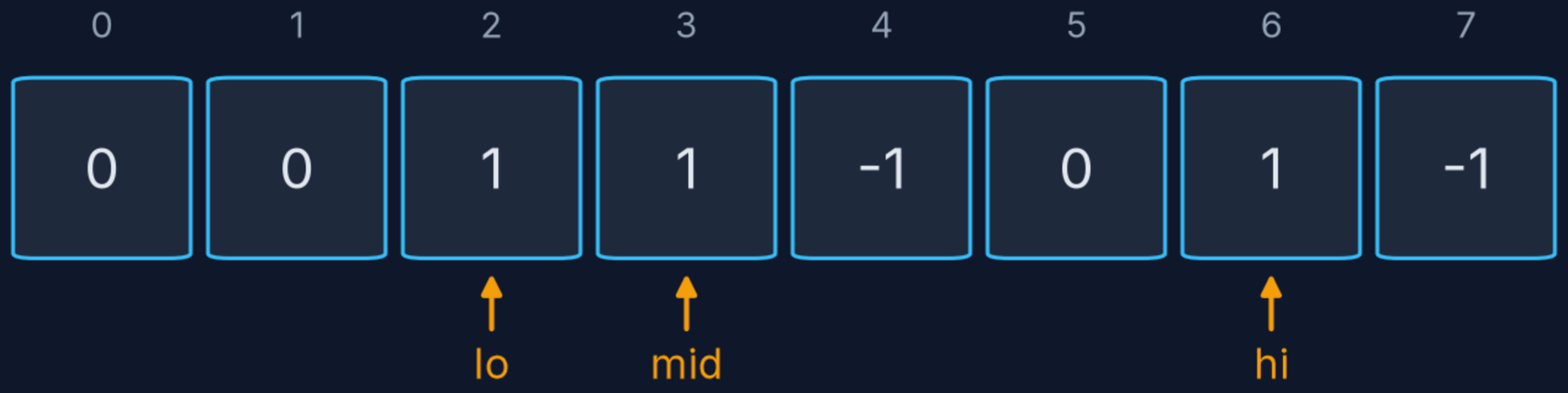
Worked example




```
saw 0: first → swap to lo
```

0: first → swap to lo

Worked example




```
saw -1: third → swap to hi
```

-1: third → swap to hi

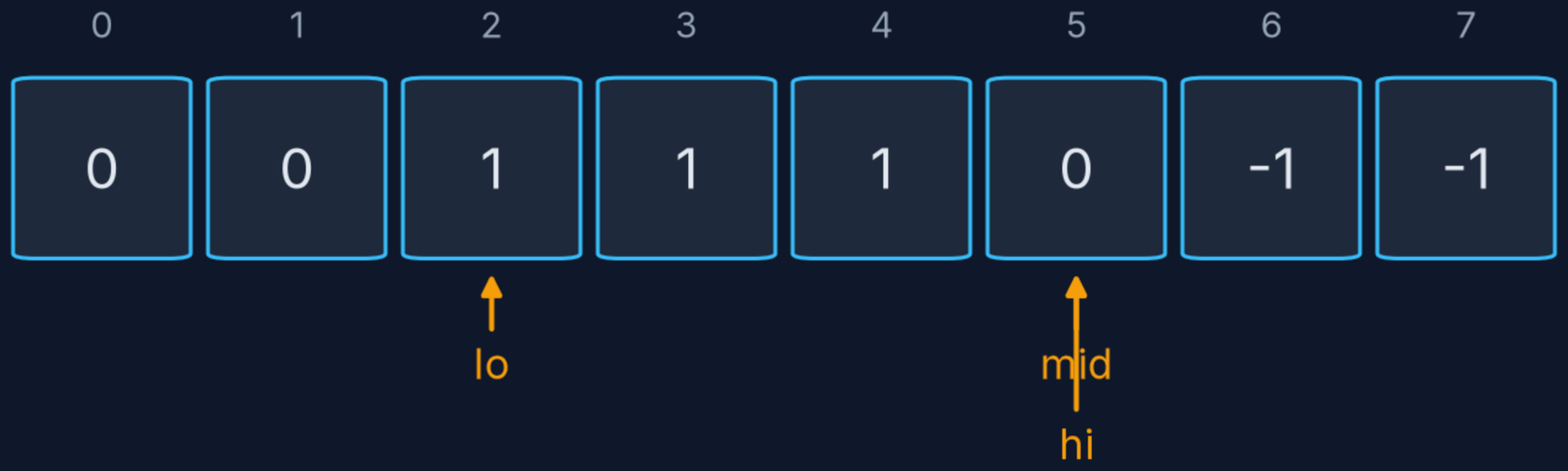
Worked example




```
saw 1: second → leave it
```

1: second → leave it

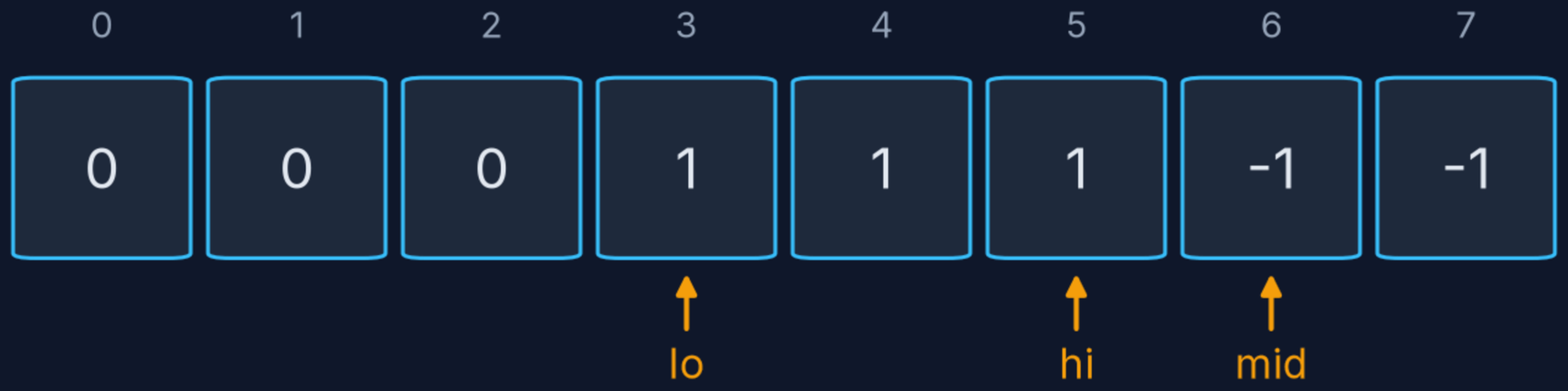
Worked example




```
saw 1: second → leave it
```

1: second → leave it

Worked example




```
saw 0: first → swap to lo
```

0: first → swap to lo

Java solution

```java
public class ThreeNumberSort {
    public static int[] threeNumberSort(int[] a, int[] order) {
        int lo = 0, mid = 0, hi = a.length - 1;
        while (mid <= hi) {
            if (a[mid] == order[0]) swap(a, lo++, mid++);
            else if (a[mid] == order[1]) mid++;
            else swap(a, mid, hi--);
        }
        return a;
    }

    private static void swap(int[] a, int i, int j) { int t = a[i]; a[i] = a[j]; a[j] = t; }
}
```

Complexity

- Time: O(n)
- Space: O(1)

Edge cases & pitfalls

- Don't advance mid after swapping with hi
- Values may be missing
- Loop while mid ≤ hi

Interview tips & follow-ups

- Name the Dutch national flag problem

Recap

- Dutch national flag: three regions grown by three pointers in one pass.
- Complexity: O(n)